C-এর ডেটা টাইপ ও অপারেটর

সাইজ · রূপান্তর · অগ্রাধিকার · if, switch, লুপ · ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো লেসন এক পাতায়: লাল = বেশি আসে, সবুজ = যে পরিবার থেকে প্রায় সব প্রশ্ন আসে

➜ সিলেবাসের যে অংশ

সিলেবাসের লাইন "*Structured programming language: data types, operators, expressions, control structures*"। ফাংশন, স্কোপ, স্টোরেজ ক্লাস ও রিকারশন; প্রিপ্রসেসর, পয়েন্টার, অ্যারে ও স্ট্রিং; স্ট্রাকচার, ইউনিয়ন ও *enum*; এবং *I/O* ও ফাইল: এই অধ্যায়ে প্রতিটির আলাদা লেসন আছে।

হাই-ইল্ড তালিকা

① ৩২টি কিওয়ার্ড (C89)
② int 4 B · char 1 B
③ −7/2 = −3 · −7%2 = −1
④ float f = 7/2 → 3.0
⑤ −1 < 1u মিথ্যা (0)
⑥ b = a++ পুরোনো a নেয়
⑦ && ও || শর্ট-সার্কিট করে
⑧ sizeof অপারেন্ড চালায় না
⑨ switch: break নেই ⇒ ফল-থ্রু
⑩ do-while অন্তত একবার চলে

সংকেত হলুদ = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

➜ সূচিপত্র

✓ প্রোগ্রামের গঠন ও টোকেন ... পৃ. ২
✓ ডেটা টাইপ, সাইজ ও রেঞ্জ ... পৃ. ৩
✓ টাইপ রূপান্তর ও ওভারফ্লো ... পৃ. ৪
✓ অপারেটর ১: গাণিতিক থেকে ++ ... পৃ. ৫
✓ অপারেটর ২: বিটওয়াইজ, ?:, sizeof ... পৃ. ৬
✓ অগ্রাধিকার ও এক্সপ্রেশন ... পৃ. ৭
✓ সিদ্ধান্ত: if ও switch ... পৃ. ৮
✓ লুপ ও জাম্প স্টেটমেন্ট ... পৃ. ৯
✓ রাতের পাতা পৃ. ১০ · অনুশীলন পৃ. ১১

১ C প্রোগ্রামের গঠন ও টোকেন ভিত্তি ★

সংজ্ঞা C একটি স্ট্রাকচার্ড (প্রসিডিউরাল) ভাষা: প্রোগ্রাম হলো কতগুলো ফাংশনের সমষ্টি, এক্সিকিউশন সবসময় `main()` থেকে শুরু হয়, আর নিয়ন্ত্রণ চলে কেবল **ক্রম (sequence), নির্বাচন (selection) ও পুনরাবৃত্তি (iteration)** দিয়ে — স্ট্রাকচার্ড প্রোগ্রামিংয়ের তিন কাঠামো (Böhm–Jacopini)। কম্পাইলার প্রথমে সোর্সকে টোকেনে (Token) ভাগ করে — অর্থবহ ক্ষুদ্রতম একক।

/* ডকুমেন্টেশন কমেন্ট */

#include · #define (প্রিপ্রসেসর)

গ্লোবাল ডিক্লারেশন

int main(void) { ... }
আগে ডিক্লারেশন, পরে স্টেটমেন্ট

ইউজার-ডিফাইন্ড ফাংশন

ওপর → নিচ = সাধারণ বিন্যাস

```c
#include <stdio.h>   /* header file     */
int global = 100;    /* global variable */
int main(void) {     /* starts here     */
    int local = 10;  /* local variable  */
    printf("Sum: %d\n", global + local);
    return 0;        /* 0 = success     */
}
```

আউটপুট: **Sum: 110**। স্ট্রিংয়ের ভেতরের `\n` একটি এস্কেপ সিকোয়েন্স (Escape sequence) — একটিমাত্র অক্ষর, নতুন লাইন। `return 0;` অপারেটিং সিস্টেমকে সফলতার খবর দেয়।

পাঁচ অংশ; কেবল `main()` বাধ্যতামূলক

টোকেনের ধরন	এটি কী	উদাহরণ
কিওয়ার্ড (Keyword)	নির্দিষ্ট অর্থের সংরক্ষিত শব্দ, সব **ছোট হাতের**	`int`, `while`, `sizeof`, `return`
আইডেন্টিফায়ার	ভেরিয়েবল, ফাংশন বা লেবেলের জন্য তোমার দেওয়া নাম	`sum`, `_total`, `area2`
কনস্ট্যান্ট	সোর্সে লেখা স্থির মান	`25`, `031`, `0x19`, `3.14f`, `'A'`
স্ট্রিং লিটারেল	ডাবল কোটে অক্ষর, শেষে `'\0'` যোগ হয়	`"Dhaka"` (6 bytes)
অপারেটর / পাংচুয়েটর	অপারেন্ডে কাজ করে / আলাদা করে	`+ ++ && ?: · ; , { } ( )`

কিওয়ার্ড C89/C90: ৩২টি · C99 যোগ করে ৫টি (`inline`, `restrict`, `_Bool`, `_Complex`, `_Imaginary`) = ৩৭ · C11 আরও ৭টি = ৪৪। কিওয়ার্ড **নয়**: `main`, `printf`, `include`, `define`, `NULL`, `string`, `boolean`; `bool`/`true` আসে `<stdbool.h>` থেকে (C23-এর আগে কিওয়ার্ড নয়)।

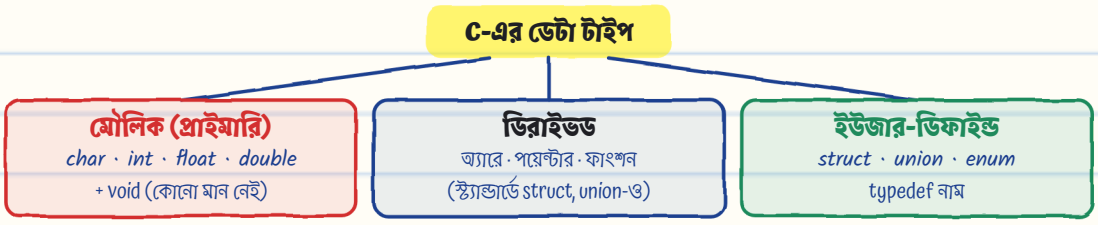
আইডেন্টিফায়ারের নিয়ম কেবল অক্ষর, অঙ্ক ও `_`; প্রথমে **অঙ্ক চলবে না**; স্পেস বা চিহ্ন নয় (`my-var`, `$x` X); কিওয়ার্ড নয়; কেস-সেনসিটিভ (`Sum` ≠ `sum`)। অন্তত ৩১টি অক্ষর অর্থবহ (C89), C99-এ ৬৩টি।

মনে রাখার কৌশল কনস্ট্যান্টের উপসর্গ: "**0 = অক্টাল, 0x = হেক্স**"। `25` = `031` = `0x19`। প্রত্যয়: `U` unsigned, `L` long, `LL` long long, `f` float। শুধু `3.14` লিখলে তা double।

পরীক্ষার ফাঁদ `int x = 010;` ছাপে **8**, 10 নয় (অক্টাল)। C-তে `'A'` হলো `int` টাইপের ক্যারেক্টার কনস্ট্যান্ট, তাই `sizeof('A') = 4` (C++-এ 1); `"A"` স্ট্রিং: `sizeof("A") = 2`। `089` ভুল — 8, 9 অক্টাল অঙ্ক নয়।

২ ডেটা টাইপ, সাইজ ও রেঞ্জ সবচেয়ে বেশি আসে ★

সংজ্ঞা ডেটা টাইপ (Data type) একটি ভেরিয়েবলের তিনটি বিষয় ঠিক করে: কত বাইট জায়গা নেবে, কোন **রেঞ্জের** মান রাখতে পারবে, আর তার ওপর কোন **অপারেশন** বৈধ। স্ট্যান্ডার্ড কেবল **ন্যূনতম** সাইজ বেঁধে দেয়; আসল সাইজ কম্পাইলার ও প্ল্যাটফর্মভেদে আলাদা — সবসময় `sizeof` দিয়ে যাচাই করো।



মডিফায়ার *short, long, signed, unsigned* মৌলিক টাইপকে বদলায়; *struct* ও *union*-কে ডিরাইভড অথবা ইউজার-ডিফাইন্ড বলা হয় — পরীক্ষায় দুটোই দেখা যায়

টাইপ	সাধারণ সাইজ	রেঞ্জ (two's complement)	printf
char / signed char	1 B	−128 … 127	%c, %d
unsigned char	1 B	0 … 255	%c, %u
short	2 B	−32,768 … 32,767	%hd
int	4 B	−2,147,483,648 … 2,147,483,647	%d, %i
unsigned int	4 B	0 … 4,294,967,295	%u, %x, %o
long	4 বা 8 B	অন্তত ±2,147,483,647	%ld
long long	8 B	≈ $\pm 9.22 \times 10^{18}$	%lld
float	4 B	≈ $\pm 3.4 \times 10^{38}$, **6–7** অঙ্ক	%f, %e, %g
double	8 B	≈ $\pm 1.7 \times 10^{308}$, **15–16** অঙ্ক	%f (scanf %lf)
long double	8–16 B	≥ double	%Lf

ডেটা মডেল	int	long	pointer
16-bit (Turbo C)	2	4	2
ILP32 (32-bit)	4	4	4
LP64 Linux/macOS	4	**8**	8
LLP64 Windows	4	4	8

স্ট্যান্ডার্ডের নিশ্চয়তা *sizeof(char) = 1* সবসময় · 1 ≤ short ≤ int ≤ long ≤ long long · short, int ≥ 16 bit, long ≥ 32, long long ≥ 64 · n বিটের রেঞ্জ: signed $-2^{n-1} \ldots 2^{n-1}-1$, unsigned $0 \ldots 2^n-1$ · সীমা <limits.h> (INT_MAX) ও <float.h>-এ (FLT_DIG = 6)।

সমাধানকৃত উদাহরণ ✍ char আসলে পূর্ণসংখ্যা

```
char c = 'A'; printf("%d %c", c, c + 2);
```

→ **65 C**: `c + 2` হলো int 67, যা অক্ষর হিসেবে ছাপা হয়। `'0' = 48` এবং `'a' − 'A' = 32`।

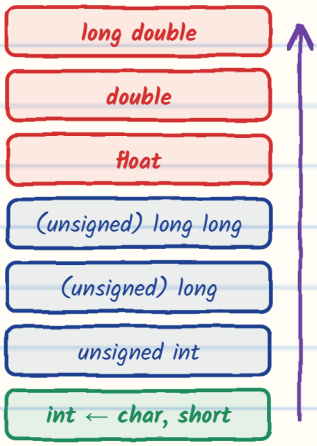
পরীক্ষার ফাঁদ `float f = 0.1;` এরপর `f == 0.1` **মিথ্যা**: 0.1 একটি double, float কপিটি কম বিটে রাউন্ড হয় (0.5 বাইনারিতে নিখুঁত, তাই সত্য)। তুলনা করো `fabs(a - b) < 1e-6` দিয়ে।

মনে রাখার কৌশল "**1-2-4-8, 4-8**": char, short, int, long long; float, double। 16-বিট Turbo C-তে int ছোট হয়ে 2 B, তাই "16-বিট কম্পাইলারে int-এর রেঞ্জ" = −32,768 … 32,767।

৩ টাইপ রূপান্তর, কাস্টিং ও ওভারফ্লো

আউটপুট প্রশ্ন ★

সংজ্ঞা ইমপ্লিসিট রূপান্তর *(Implicit conversion)* কম্পাইলার নিজে করে: মিশ্র এক্সপ্রেশনে, অ্যাসাইনমেন্টে ও আর্গুমেন্ট পাঠানোর সময়। এক্সপ্লিসিট রূপান্তর হলো **কাস্ট** *(Cast)*, লেখা হয় `(type) expression`। কোনো অপারেশনের টাইপ ঠিক হয় তার **অপারেন্ড** দিয়ে, ফল যে ভেরিয়েবলে যাবে তা দিয়ে কখনো নয়।



নিচের অপারেন্ড ওপরে ওঠে

সাধারণ গাণিতিক রূপান্তর

১. প্রথমে **ইন্টিজার প্রমোশন** *(Integer promotion)*: যেকোনো গাণিতিক কাজের আগে *char, short (signed বা unsigned)* ও *_Bool* হয়ে যায় *int*। তাই `'A' + 1` একটি *int*।

২. কোনো অপারেন্ড ***long double / double / float*** হলে অন্যটিও সেই টাইপে রূপান্তরিত হয়।

৩. নাহলে নিচু **র‍্যাঙ্কের** অপারেন্ড উঁচুটিতে ওঠে; সমান র‍্যাঙ্কে *unsigned* জেতে।

৪. **অ্যাসাইনমেন্ট** মানটিকে বাম পাশের টাইপে বদলায়: *float → int* **শূন্যের দিকে কাটে** *(3.99 → 3, −3.99 → −3)*।

এক্সপ্রেশন	মান	কারণ
`float f = 7 / 2;`	3.0	*int ÷ int = 3*, পরে *float*
`(float)7 / 2`	3.5	আগে কাস্ট, *2 → float*
`(float)(7 / 2)`	3.0	*int* ভাগের পরে কাস্ট
`5 / 2 * 2.0`	4.0	আগে *5/2 = 2* (বাম থেকে ডান)
`int i = 3.99;`	3	ভগ্নাংশ বাদ

ওভারফ্লো (Overflow) *Unsigned* গণিত কখনো ওভারফ্লো করে না — এটি *2ⁿ* মডুলোতে ঘুরে আসে: `unsigned char u = 255; u++;` → *0*; `unsigned x = 0; x--;` → *4,294,967,295*। ***Signed*** ওভারফ্লো *(INT_MAX + 1)* **অনির্ধারিত আচরণ**; বেশিরভাগ মেশিনে ঘটনাচক্রে *INT_MIN*-এ ঘুরে যায়। *signed char*-এ রেঞ্জের বাইরের মান রাখা ইমপ্লিমেন্টেশন-নির্ভর: `char c = 200;` সাধারণত ***−56*** *(200 − 256)*।

পরীক্ষার ফাঁদ: signed বনাম unsigned

```
int i = -1;  unsigned u = 1;
if (i < u) printf("less");
else       printf("greater");
```

ছাপে ***greater***: *−1 unsigned*-এ রূপান্তরিত হয়ে *4,294,967,295*। একই কারণে `sizeof(int) > -1` **মিথ্যা** *(sizeof দেয় unsigned size_t)*। তবে `unsigned char` ও `unsigned short` আগে *int*-এ প্রমোট হয়, তাই *−1 < (unsigned char)1* সত্য।

মনে রাখার কৌশল "**অপারেশন হয় অপারেন্ডের টাইপে; ফল রূপান্তরিত হয় কেবল নামার সময়।**" আগে ডান পাশ দেখো: দুটোই *int* ⇒ পূর্ণসংখ্যার ভাগ, বামে যে টাইপই অপেক্ষা করুক।

পরীক্ষার ফাঁদ `printf("%f", 5 / 2);` *%f*-এ একটি ***int*** পাঠায় — অনির্ধারিত আচরণ (প্রায়ই *0.000000* ছাপে), *2.5* নয়। `printf("%d", 2.5);`-ও সমান ভুল। স্পেসিফায়ার টাইপের সঙ্গে মেলাও।

৪ অপারেটর ১: গাণিতিক থেকে ++/-- মূল ★

শ্রেণি	অপারেটর	ফল / মন্তব্য
গাণিতিক (Arithmetic)	+ - * / %	% শুধু পূর্ণসংখ্যায় (float-এ fmod())
সম্পর্কসূচক (Relational)	< <= > >= == !=	int 1 (সত্য) বা 0 (মিথ্যা)
যৌক্তিক (Logical)	&& || !	1 বা 0; যেকোনো অশূন্য মান সত্য
অ্যাসাইনমেন্ট	= += -= *= /= %= &= |= ^= <<= >>=	a *= b + 1 মানে a = a * (b + 1)
ইনক্রিমেন্ট/ডিক্রিমেন্ট	++ -- (প্রিফিক্স ও পোস্টফিক্স)	কেবল পরিবর্তনযোগ্য ভেরিয়েবলে (lvalue)

পূর্ণসংখ্যার / ও % (C99 নিয়ম) ভাগফল শূন্যের দিকে কাটা হয়; ভাগশেষ পায় **ভাজ্যের চিহ্ন**, যাতে (a / b) * b + a % b == a সবসময় সত্য থাকে।

a, b	a / b	a % b
17, 5	3	2
-7, 2	-3	-1
7, -2	-3	1
-7, -2	3	-1

শূন্য দিয়ে পূর্ণসংখ্যার ভাগ অনির্ধারিত (সাধারণত ক্র্যাশ)।

সমাধানকৃত উদাহরণ ✍ শর্ট-সার্কিট

```
int a = 0, b = 5;
if (a && ++b) ;   /* ++b skipped */
if (a || ++b) ;   /* ++b runs    */
printf("%d", b);  /* 6           */
```

&& প্রথম মিথ্যা অপারেন্ডে থামে, || প্রথম সত্যটিতে। চিরায়ত প্রশ্ন: i = 1, j = 0; k = i++ || j++; → i = 2, j = 0, k = 1, কারণ i++ দিয়েছে 1 (সত্য), তাই j++ চলেইনি।

++a (প্রিফিক্স)
a = a+1 → তারপর নতুন a

a++ (পোস্টফিক্স)
পুরোনো a দাও → তারপর a = a+1

সমাধানকৃত উদাহরণ ✍ প্রি বনাম পোস্ট

```
int a = 5, b, c;
b = a++;   /* b = 5, then a = 6 */
c = ++a;   /* a = 7, then c = 7 */
printf("%d %d %d", a, b, c);
```

আউটপুট: 7 5 7। আলাদা স্টেটমেন্ট হিসেবে a++; ও ++a; একই কাজ করে।

দুই ক্ষেত্রেই a এক বাড়ে; শুধু এক্সপ্রেশনকে দেওয়া মানটি আলাদা

পরীক্ষার ফাঁদ (i) 10 > 5 > 2-এর মান 0: (10 > 5) = 1, তারপর 1 > 2 = 0। (ii) !5 = 0, !!7 = 1। (iii) 5++, (a + b)++ কম্পাইল এরর — এরা lvalue নয়। (iv) x = x++; অনির্ধারিত, পাতা ৭ দেখো।

মনে রাখার কৌশল অবস্থান পড়ো: "**++ আগে = আগে বদল; ++ পরে = পরে বদল**।" যুক্তির জন্য: "**AND থামে একটি শূন্যে, OR থামে একটি অশূন্যে**" — তার পরের অংশ আর মূল্যায়িত হয় না।

৫ অপারেটর ২: বিটওয়াইজ, শিফট, ?:, কমা, sizeof

বিট কৌশল ★

সংজ্ঞা বিটওয়াইজ অপারেটর (*Bitwise operator*) **পূর্ণসংখ্যা** অপারেন্ডের প্রতিটি বিটে কাজ করে: `&` (AND), `|` (OR), `^` (XOR), `~` (ওয়ান'স কমপ্লিমেন্ট, ইউনারি), `<<` (বাম শিফট), `>>` (ডান শিফট)। *float* বা *double*-এ এগুলো চলে না।

`a = 12` → `1100`
`b = 10` → `1010`

`a & b`	`1000`	= 8
`a | b`	`1110`	= 14
`a ^ b`	`0110`	= 6
`a >> 2`	`0011`	= 3

12 ও 10 বিট ধরে ধরে; `a << 2` = 48, `~a` = −13

নিয়ম	তথ্য
`x << n`	$x \times 2^n$ (ওভারফ্লো না হলে)
`x >> n`	$x \div 2^n$, ফ্লোর, $x \geq 0$ হলে
`~x` (int)	$-x - 1$; `~5` = −6, `~0` = −1
ঋণাত্মক `>>`	ইমপ্লিমেন্টেশন-নির্ভর
শিফট ≥ প্রস্থ	অনির্ধারিত আচরণ

বিট কৌশল `x & 1` → বিজোড় যাচাই · `x & (x - 1) == 0` → 2-এর ঘাত (`x > 0`) · k-তম বিট সেট: `x | (1 << k)` · ক্লিয়ার: `x & ~(1 << k)` · টগল: `x ^ (1 << k)` · অদলবদল: `a ^= b; b ^= a; a ^= b`।

শর্তসাপেক্ষ (Ternary) ?: `c ? e1 : e2` — তিন অপারেন্ডের একমাত্র অপারেটর; *e1*, *e2*-এর **কেবল একটি** মূল্যায়িত হয়। ডান থেকে বামে যুক্ত হয়: `a > b ? (a > c ? a : c) : (b > c ? b : c)` তিনটির বৃহত্তম দেয়। `printf("%s", 7 % 2 ? "odd" : "even");` → *odd*।

কমা অপারেটর (Comma) বাম থেকে ডানে মূল্যায়ন করে; মান হয় সবচেয়ে ডানের অপারেন্ড; এর অগ্রাধিকার **সর্বনিম্ন**। `x = (1, 2, 3);` → `x = 3`, কিন্তু `x = 1, 2, 3;` → `x = 1` (= আগে যুক্ত হয়)। ফাংশন আর্গুমেন্ট বা ডিক্লারেশনের কমা বিভাজক, এই অপারেটর নয়।

সমাধানকৃত উদাহরণ ✍ sizeof মূল্যায়ন করে না

```
int i = 5;
int s = sizeof(i++);
printf("%d %d", i, s);   /* 5 4 */
```

sizeof একটি **কম্পাইল-টাইম ইউনারি অপারেটর** (এবং কিওয়ার্ড): এর শুধু টাইপ লাগে, তাই `i++` চলে না। ফলের টাইপ *size_t*, ছাপো **%zu** দিয়ে। বন্ধনী লাগে কেবল টাইপের নামে: `sizeof x, sizeof(int)`।

পরীক্ষার ফাঁদ (i) যৌক্তিক বনাম বিটওয়াইজ: `2 && 1` = 1 কিন্তু `2 & 1` = 0। (ii) `^` হলো XOR, ঘাত **নয়** — `2 ^ 3` = 1; ঘাতের জন্য `pow()`। (iii) `sizeof(3.14)` = 8 (*double*), `sizeof(3.14f)` = 4। (iv) পয়েন্টার অপারেটর `&` ও `*`, এবং মেম্বার অপারেটর `.` ও `->` পয়েন্টার ও স্ট্রাকচার লেসনে আছে।

সমাধানকৃত উদাহরণ ✍ 1-বিট গোনা

```
int n = 13, c = 0;   /* 1101 */
while (n) {
    n &= n - 1;      /* drop lowest 1 */
    c++;
}
printf("%d", c);     /* 3 */
```

13 = 1101 → 1100 → 1000 → 0000: প্রতিটি সেট বিটে লুপ একবার চলে, তাই ছাপে **3**। একই **`n & (n - 1)`** ধারণায় 2-এর ঘাত যাচাই হয়। ঋণাত্মক সংখ্যা রাখা হয় টু'স কমপ্লিমেন্টে (*Two's complement*): `-x = ~x + 1`, তাই 8 বিটে −13 = 1111 0011।

৬ অগ্রাধিকার, অ্যাসোসিয়েটিভিটি ও এক্সপ্রেশন MCQ-এর প্রিয় ★

#	অপারেটর (উঁচু → নিচু)	দিক
1	`()` `[]` `->` `.` postfix `++` `--`	L → R
2	unary `+` `-` `!` `~` prefix `++` `--` `*` `&` `sizeof` (type)	R → L
3	`*` `/` `%`	L → R
4	`+` `-`	L → R
5	`<<` `>>`	L → R
6	`<` `<=` `>` `>=`	L → R
7	`==` `!=`	L → R
8–10	`&`, তারপর `^`, তারপর `|`	L → R
11–12	`&&`, তারপর `||`	L → R
13	`?:`	R → L
14	`=` `+=` `-=` `*=` `/=` `%=` `&=` `^=` `|=` `<<=` `>>=`	R → L
15	`,` (কমা)	L → R

মনে রাখার কৌশল "PUMA'S REBL TAC": Postfix, Unary, Multiplicative, Additive, Shift, Relational, Equality, Bitwise (& ^ |), Logical (&& ||), Ternary, Assignment, Comma। কেবল **ইউনারি, টারনারি, অ্যাসাইনমেন্ট** ডান থেকে বামে যুক্ত হয়।

সমাধানকৃত উদাহরণ ✍ মান বের করো

(ক) `2 + 3 * 4 % 5 - 6 / 4` → `12 % 5 = 2, 6 / 4 = 1` → **3**। **(খ)** `a = 10, b = 20, c = 30; a + b * c / 10 % 7` → `600 / 10 = 60, 60 % 7 = 4` → **14**। **(গ)** `1 << 2 + 1` → আগে `+` → `1 << 3 = 8`। **(ঘ)** `5 & 3 | 4` → `(1) | 4 = 5`। **(ঙ)** `a = b = c = 5` → ডান থেকে বামে, সবাই **5**।

পরীক্ষার ফাঁদ `if (x & 1 == 0)` পার্স হয় **`x & (1 == 0)`** = `x & 0` = **সবসময় 0** — বিটওয়াইজ অপারেটর `==` এর **নিচে**, তাই লেখো `(x & 1) == 0`। একইভাবে `*p++` মানে `*(p++)`, আর `!a + b` মানে `(!a) + b`।

এক্সপ্রেশন · lvalue · rvalue এক্সপ্রেশন (Expression) হলো অপারেটর দিয়ে জোড়া অপারেন্ড, যা একটি মান দেয় (গাণিতিক, সম্পর্কসূচক, যৌক্তিক, শর্তসাপেক্ষ, অ্যাসাইনমেন্ট)। **lvalue** একটি মেমোরি অবস্থান নির্দেশ করে, `=` এর বামে বসতে পারে; **rvalue** কেবল একটি মান। `5 = x`, `a + b = c`, `x++ = 3` এরর। অ্যাসাইনমেন্ট নিজেও এক্সপ্রেশন: `while ((ch = getchar()) != EOF)`।

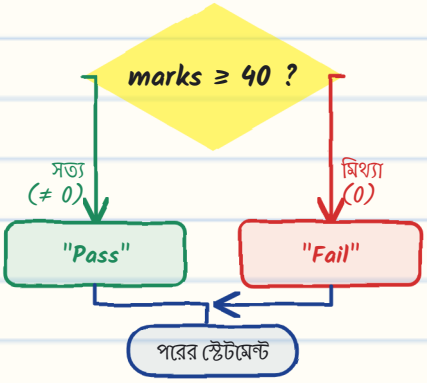
সিকোয়েন্স পয়েন্ট ও অনির্ধারিত আচরণ মাঝে কোনো সিকোয়েন্স পয়েন্ট (Sequence point) ছাড়া একটি ভেরিয়েবল **দুবার** বদলানো, বা বদলানোর সঙ্গে পড়াও, **অনির্ধারিত**। `i = i++;` · `i = ++i + i++;` · `a[i] = i++;` · `printf("%d %d", i, i++);`। সিকোয়েন্স পয়েন্ট: পূর্ণ এক্সপ্রেশনের শেষ (`;`), `&&`, `||`, `?:` ও কমা অপারেটরের বাম অপারেন্ডের পরে, এবং ফাংশন কলের ঠিক আগে।

পরীক্ষার ফাঁদ **অগ্রাধিকার মানে মূল্যায়নের ক্রম নয়।** `f() + g() * h()`-এ গুণটি আগে গ্রুপ হয়, কিন্তু কম্পাইলার `f`, `g`, `h` যেকোনো ক্রমে **কল** করতে পারে; ফাংশন আর্গুমেন্টের মূল্যায়ন-ক্রমও অনির্দিষ্ট। `i = i++ + ++i`-এর MCQ উত্তর "অনির্ধারিত / কম্পাইলার-নির্ভর"। `gcc -Wall` সতর্ক করে "operation on 'i' may be undefined"।

৭ সিদ্ধান্ত গ্রহণ: if, else-if ল্যাডার, switch

নির্বাচন ★

সংজ্ঞা সিলেকশন স্টেটমেন্ট (Selection statement) কয়েকটি পথ থেকে একটি বেছে নেয়। শর্ত যেকোনো স্কেলার এক্সপ্রেশন: **শূন্য = মিথ্যা, যেকোনো অশূন্য মান (এমনকি −1) = সত্য**। রূপ: **if**, **if-else**, **else-if ল্যাডার**, **নেস্টেড if**, এবং একটি পূর্ণসংখ্যার মানে অনেক সমতা-যাচাইয়ের জন্য **switch**।



if-else: ঠিক একটি শাখা চলে

```
if (marks >= 80)      printf("A");
else if (marks >= 60) printf("B");
else if (marks >= 40) printf("C");
else                  printf("F");
```

else-if ল্যাডার ওপর থেকে নিচে যাচাই হয় এবং **প্রথম সত্য** শর্তে থেমে যায়, তাই শর্তগুলো কঠোরতম থেকে নিচের দিকে সাজাও: *marks = 85* হলে কেবল **A** ছাপে।

সমাধানকৃত উদাহরণ ✍ ড্যাংলিং else

```
int a = 5, b = 10;
if (a > 10)
    if (b > 5) printf("X");
else printf("Y");
```

কিছুই ছাপে না; *else* যুক্ত হয় **সবচেয়ে কাছের জোড়াহীন *if*-এর** সঙ্গে, ইনডেন্টেশন যা-ই বলুক। ব্রেস ব্যবহার করো।

পরীক্ষার ফাঁদ *(i)* `if (x = 0) 0` **অ্যাসাইন** করে → সবসময় মিথ্যা; `if (x = 10)` → সবসময় সত্য, আর x হয় 10। *(ii)* `if (x == 5);` — বাড়তি সেমিকোলনটি ফাঁকা বডি, তাই পরের লাইন সবসময় চলে। *(iii)* `5 == x` লিখলে টাইপো `5 = x` কম্পাইল এরর দেয়।

switch-এর নিয়ম এক্সপ্রেশনটি হতে হবে পূর্ণসংখ্যা টাইপ (*int, char, enum*) — *float* বা স্ট্রিং নয়। *case* লেবেল **পূর্ণসংখ্যার কনস্ট্যান্ট এক্সপ্রেশন**, অনন্য, রেঞ্জ নয়। ***default*** ঐচ্ছিক, যেকোনো জায়গায় বসতে পারে। ***break*** না থাকলে নিয়ন্ত্রণ পরের *case*-এ **ফল-থ্রু করে**।

সমাধানকৃত উদাহরণ ✍ ফল-থ্রু

```
int n = 2;
switch (n) {
  case 1: printf("A");
  case 2: printf("B");
  case 3: printf("C"); break;
  default: printf("D");
}
```

case 2-এ লাফ দেয়, *B* ছাপে, *case 3*-এ গড়িয়ে *C* ছাপে, তারপর *break* → **BC**।

সমাধানকৃত উদাহরণ ✍ প্রথমে default

```
int n = 5;
switch (n) {
  default: printf("D");
  case 1:  printf("A"); break;
  case 2:  printf("B");
}
```

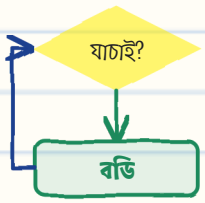
কোনো *case* মেলে না, তাই নিয়ন্ত্রণ যায় *default*-এ, **সেটি যেখানেই থাকুক**: *D*, তারপর *case 1*-এ গড়িয়ে *A*, *break* → **DA**।

মনে রাখার কৌশল "**switch হলো লিফট, সিঁড়ি নয়**": সরাসরি মিলে যাওয়া তলায় যায়, তারপর ***break*** না থামানো পর্যন্ত নিচে নামতেই থাকে। একটি পূর্ণসংখ্যাকে কনস্ট্যান্টের সঙ্গে তুলনায় **switch** (প্রায়ই দ্রুত জাম্প টেবিলে কম্পাইল হয়); রেঞ্জ, *float*, স্ট্রিং বা মিশ্র শর্তে ***if* ল্যাডার**।

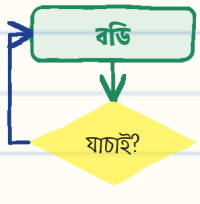
৮ লুপ (পুনরাবৃত্তি) ও জাম্প স্টেটমেন্ট

আউটপুট ট্রেসিং ★

লুপ	যাচাই	বডি চলে
`while (c) S`	শুরুতে	০ বা তার বেশি বার
`for (i; c; u) S`	শুরুতে	০ বা বেশি; অংশগুলো ঐচ্ছিক, ফাঁকা `c` = সত্য
`do S while (c);`	শেষে	**অন্তত একবার**



while: আগে যাচাই



do-while: আগে বডি

এন্ট্রি বনাম এক্সিট নিয়ন্ত্রণ

সমাধানকৃত উদাহরণ ✍ আউটপুট ট্রেস করো

```
/* (a) */ for (i = 0; i < 5; i++);
          printf("%d", i);          /* 5    */
/* (b) */ for (i = 1; i <= 100; i *= 2)
              count++;              /* 7    */
/* (c) */ i = 10;
          do printf("%d", i);
          while (i < 5);            /* 10   */
/* (d) */ n = 3;
          while (n--) printf("%d", n);
                                    /* 210  */
```

```
/* (e) */ i = 0;
while (i < 5) {
    if (i == 2) { i++; continue; }
    printf("%d", i);  i++;
}                                   /* 0134 */
/* (f) */
for (i = 1; i <= 3; i++)
  for (j = 1; j <= 3; j++) {
    if (j == 2) break;
    printf("%d%d ", i, j);
  }                                 /* 11 21 31 */
```

(a) `for`-এর পরের `;` একটি ফাঁকা বডি, তাই শুধু শেষ `i = 5` ছাপা হয়। (b) `i` = 1, 2, 4, 8, 16, 32, 64 → **৭ বার চলে**। (c) `10 < 5` মিথ্যা হলেও `do-while` একবার ছাপে। (d) যাচাই হয় পুরোনো `n` দিয়ে, বডি দেখে নতুনটি; শেষে `n = −1`। (f) **`break`** কেবল **সবচেয়ে ভেতরের** লুপ ছাড়ে।

জাম্প	প্রভাব	সাবধান
`break`	সবচেয়ে ভেতরের লুপ বা `switch` থেকে বের হয়	শুধু `if`-এর ভেতরে (লুপ ছাড়া) চলে না
`continue`	পরের পুনরাবৃত্তিতে যায় (শুধু লুপে)	`for`-এ যায় **আপডেটে**; `while`-এ **যাচাইয়ে**
`goto label`	একই ফাংশনের ভেতরে শর্তহীন লাফ	বৈধ ব্যবহার: নেস্টেড লুপ থেকে বের হওয়া, ক্লিনআপ
`return`	ফাংশন ছেড়ে যায় (ফাংশন লেসন)	`exit()` লাইব্রেরি ফাংশন, জাম্প স্টেটমেন্ট নয়

পরীক্ষার ফাঁদ: অসীম লুপ *(i)* `while (i < 5) { if (i == 2) continue; i++; }` — `continue` `i++` এড়িয়ে যায়, তাই `i` চিরকাল 2। *(ii)* `for (unsigned u = 10; u >= 0; u--)` কখনো শেষ হয় না: `unsigned` মান **সবসময় ≥ 0**। *(iii)* `for (;;)` ও `while (1)` ইচ্ছাকৃত অসীম লুপ।

মনে রাখার কৌশল **"`do-while` হলো সেই ওয়েটার, যে আগে পরিবেশন করে, পরে জিজ্ঞেস করে।"** যোগভিত্তিক লুপে পুনরাবৃত্তি = **(শেষ − শুরু) / ধাপ + 1**: `for (i = 0; i < 10; i += 3)` → 0, 3, 6, 9 → ৪ বার। `for (int i = 0; ...)` লাগে **C99**; পুরোনো C89 কম্পাইলার মানে না।

পরীক্ষার আগের রাতের পাতা

তথ্য-পাতা

কিওয়ার্ড: ৩২ (C89), ৩৭ (C99)
আইডেন্টিফায়ার: প্রথমে অঙ্ক নয়
010 = 8 · 0x1F = 31
C-তে sizeof('A') = 4
char 1 · short 2 · int 4 · long long 8
float 4 (6–7 অঙ্ক) · double 8 (15–16)
long: LP64-এ 8, Windows-এ 4
16-bit int: −32,768 … 32,767
char, short → আগে int
সমান র‍্যাঙ্ক: unsigned জেতে
float → int শূন্যের দিকে কাটে
unsigned ঘোরে mod 2^n
−7 / 2 = −3 · −7 % 2 = −1
% এর চিহ্ন = ভাজ্যের চিহ্ন
~x = −x − 1
x << n = x × 2^n
কমা: ডানেরটির মান
sizeof: মূল্যায়ন হয় না
else → নিকটতম if
switch: শুধু পূর্ণসংখ্যা

→ MCQ-এর সেরা ১২ এক-লাইন

১. `float f = 7/2;` → **3.0**
২. `-1 < 1u` → **0** (মিথ্যা)
৩. `b = a++` নেয় **পুরোনো** a
৪. `k = i++ || j++` → j++ চলে না
৫. `x & 1 == 0` → x & 0
৬. `i = i++` → **অনির্ধারিত**
৭. কেবল unary, ?:, = ডান → বাম
৮. সর্বনিম্ন অগ্রাধিকার: **কমা**
৯. break নেই ⇒ **ফল-থ্রু**
১০. default যেকোনো জায়গায় বসে
১১. do-while: **অন্তত একবার**
১২. break ছাড়ে **ভেতরের** লুপটি

প্রধান ফাঁদ বাম পাশের টাইপ অপারেশন বদলায় না (7/2 তবুও 3) · ^ হলো XOR, ঘাত নয় · 2 && 1 = 1 কিন্তু 2 & 1 = 0 · float == float অনির্ভরযোগ্য · unsigned u >= 0 সবসময় সত্য · `if (x = 0)` অ্যাসাইন করে · for/if/while-এর পরে বাড়তি ; বডি ফাঁকা করে · অগ্রাধিকার ≠ মূল্যায়নের ক্রম · `printf("%f", 5/2)` অনির্ধারিত।

সব কৌশল একসাথে "0 = অক্টাল, 0x = হেক্স" · "1-2-4-8, 4-8" · "অপারেশন অপারেন্ডের টাইপে; ফল বদলায় নামার সময়" · "++ আগে = আগে বদল" · "AND থামে শূন্যে, OR থামে অশূন্যে" · PUMA'S REBL TAC · "switch লিফট, সিঁড়ি নয়" · "do-while আগে পরিবেশন করে, পরে জিজ্ঞেস করে"।

নির্ধারিত
unsigned ঘোরা · একা i++ · a && b++
সঠিক আউটপুট লেখো

অনির্ধারিত
i = i++ · INT_MAX + 1 · x / 0
উত্তর "কম্পাইলার-নির্ভর"

আগে বাক্স ঠিক করো, তারপর ট্রেস করো

প্রতিটি "আউটপুট কী হবে" প্রশ্নে প্রথম যা জিজ্ঞেস করতে হয়

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

আগে চেষ্টা করো, তারপর দেখো

১. gcc, 64-bit Linux-এ sizeof(int):
ক) 2 খ) 4
গ) 8 ঘ) 16

২. printf("%d", -7 / 2) ছাপে:
ক) −4 খ) 3
গ) −3 ঘ) −3.5

৩. C-তে কোনটি কিওয়ার্ড **নয়**:
ক) sizeof খ) volatile
গ) goto ঘ) string

৪. a = 5; b = a++ + 2; b কত:
ক) 7 খ) 8
গ) 6 ঘ) অনির্ধারিত

৫. -1 < 1u-এর মান:
ক) 1 খ) 0
গ) এরর ঘ) −1

৬. বডি অন্তত একবার চলে:
ক) do-while খ) for
গ) while ঘ) কোনোটিই নয়

৭. সর্বনিম্ন অগ্রাধিকার:
ক) = খ) ?:
গ) , (কমা) ঘ) ||

৮. C-তে sizeof('A') (int = 4 B):
ক) 4 খ) 1
গ) 2 ঘ) 8

৯. 5 & 3 | 4-এর মান:
ক) 7 খ) 1
গ) 4 ঘ) 5

১০. int x = 010; printf("%d", x);
ক) 10 খ) 8
গ) 2 ঘ) 16

উত্তরমালা → ১-খ · ২-গ · ৩-ঘ · ৪-ক · ৫-খ · ৬-ক · ৭-গ · ৮-ক · ৯-ঘ · ১০-খ

লিখিত প্রশ্ন (ক) ফ্লোচার্টসহ **while** ও **do-while**-এর পার্থক্য লেখো। (খ) নিচের প্রোগ্রামের আউটপুট ট্রেস করো। (গ) একটি পূর্ণসংখ্যা পড়ে তার অঙ্কগুলোর যোগফল ছাপে এমন C প্রোগ্রাম লেখো। ১০

```
int i, s = 0;
for (i = 1; i <= 5; i++)
    switch (i % 3) {
        case 0:  s += 10; break;
        case 1:  s += 1;
        default: s += 2;
    }
printf("%d", s);
```

১ **(ক)** *while* **এন্ট্রি-নিয়ন্ত্রিত**: আগে যাচাই, পরে বডি, তাই ০ বারও চলতে পারে। *do-while* **এক্সিট-নিয়ন্ত্রিত**: আগে বডি, পরে যাচাই, তাই অন্তত একবার চলে এবং শেষ হয় `while (c);` দিয়ে। ফ্লোচার্ট পাতা ৯-এর মতো। ৩

২ **(খ)** *i = 1: case 1 → +1, default-এ গড়িয়ে → +2, s = 3 · i = 2: default → 5 · i = 3: case 0 → 15 · i = 4: +1 +2 → 18 · i = 5: +2 →* **20**। আউটপুট **20**। ৩

৩ **(গ)** `int n, sum = 0; scanf("%d", &n); if (n < 0) n = -n; while (n > 0) { sum += n % 10; n /= 10; } printf("%d", sum);` — *n % 10* শেষ অঙ্কটি নেয়, *n /= 10* সেটি বাদ দেয়; *4731 → 1 + 3 + 7 + 4 = 15*। ৪

→ পরীক্ষার আগে ✓

- ✓ প্রতিটি মৌলিক টাইপের সাইজ ও রেঞ্জ বলতে পারি
- ✓ সাধারণ গাণিতিক রূপান্তর প্রয়োগ করতে পারি
- ✓ *PUMA'S REBL TAC* দিয়ে যেকোনো এক্সপ্রেশন পার্স করতে পারি
- ✓ *switch* ফল-থ্রু ও লুপ ট্রেস করতে পারি